RAG Frameworks

Hobhav S Fulzele

November 3, 2026

Contents

1 LangChain **2**
1.1 Primary Purpose . . . 2
1.2 LangChain Architecture . . . 2

2 LlamaIndex **5**
2.1 LlamaIndex Architecture . . . 5
2.2 Ingestion Pipeline . . . 7
2.3 Working Process of LlamaIndex . . . 7

3 Haystack **8**
3.1 Haystack Architecture . . . 8
3.2 Indexing Pipeline . . . 9
3.3 Query Pipeline . . . 10
3.4 Working Process of Haystack . . . 10

4 Key Differences **11**
4.1 LangChain . . . 11
4.2 LlamaIndex . . . 11
4.3 Haystack . . . 11

1 LangChain

LangChain is an open-source framework designed to simplify the development of applications powered by Large Language Models.

It provides reusable components that allow developers to connect language models, prompts, document loaders, embedding models, vector stores, retrievers and external tools.

LangChain is particularly useful for building applications that require multiple AI components to work together.

1.1 Primary Purpose

The primary purpose of LangChain is to connect different components of an LLM application through reusable interfaces and workflows.

Its important applications include:

- Retrieval-Augmented Generation.
- Conversational AI applications.
- Document question-answering systems.
- AI agents and tool integration.
- Semantic search applications.
- Automated information processing.

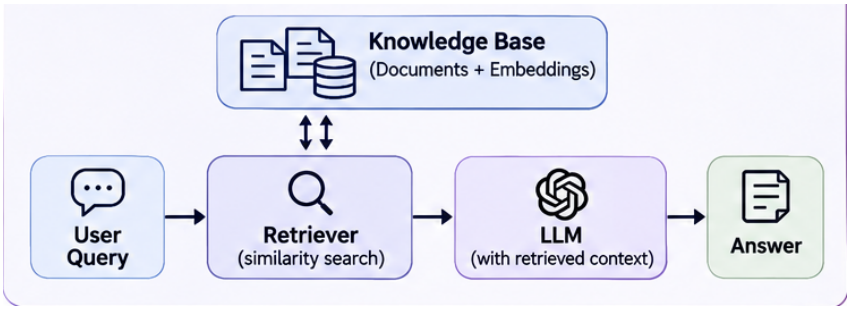


Figure 1: Rag Overview

1.2 LangChain Architecture

The general document retrieval workflow is:

$$\boxed{\textit{Document Loading} \rightarrow \textit{Text Splitting} \rightarrow \textit{Embedding} \rightarrow \textit{Retrieval}}$$

1.2.1 Document Loaders

Document loaders import information from different sources into the application.

The loaded information is represented using Document objects containing page content and metadata.

1.2.2 Text Splitters

Large documents are divided into smaller sections called chunks.

This process is known as document splitting or chunking.

The main reasons for splitting documents are:

- To reduce input size.
- To improve retrieval accuracy.
- To fit model context limitations.
- To retrieve only relevant information.

Two important parameters are:

Chunk Size: The maximum desired size of each text chunk.

Chunk Overlap: The amount of text shared between consecutive chunks.

The overlapping text helps preserve context between adjacent chunks.

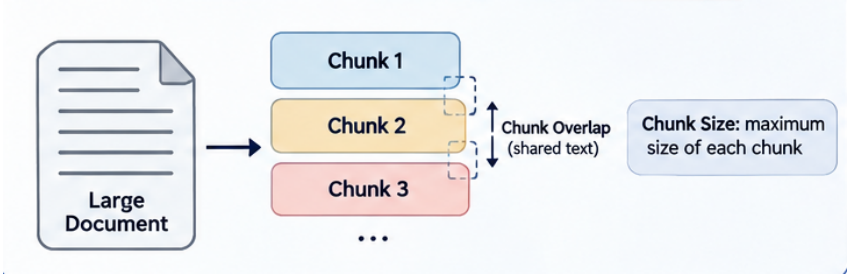


Figure 2: Document Chunking or Text Splitter

1.2.3 Embedding Models

An embedding model converts text into a numerical vector representation.

These vectors represent semantic information about the text.

Texts with similar meanings generally have similar vector representations.

Embeddings are used for semantic search and similarity-based retrieval.

1.2.4 Vector Stores

A vector store stores document embeddings along with associated document information.

Examples include:

- FAISS
- Chroma
- Pinecone
- Weaviate
- Qdrant

The vector store allows efficient similarity search over stored embeddings.

1.2.5 Retrievers

A retriever searches for relevant documents based on a user query.

It compares the query representation with stored document representations and returns the most relevant results.

Retrievers can use different search strategies:

- Dense vector search
- Keyword-based search
- Hybrid search
- Metadata filtering

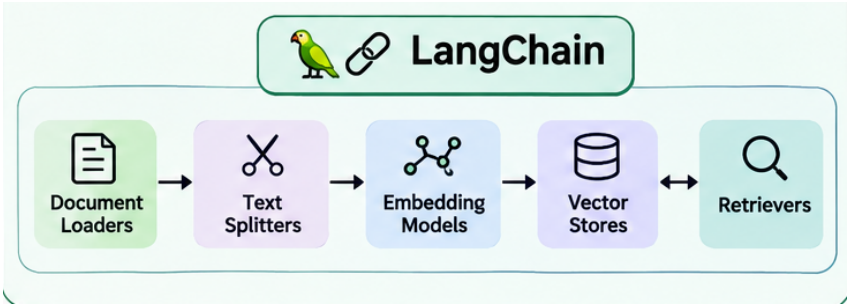


Figure 3: Langchain Overview

2 LlamaIndex

LlamaIndex is a framework designed to connect Large Language Models with external data sources.

It provides specialized tools for document ingestion, indexing, querying and retrieval.

Its main purpose is to make private and external data accessible to LLM applications.

LlamaIndex is particularly useful when an application needs to process large collections of documents and retrieve relevant information efficiently.

2.1 LlamaIndex Architecture

The general workflow is:

$$\boxed{Document \rightarrow Nodes \rightarrow Vector\ Index \rightarrow Search}$$

2.1.1 Document

A Document is the initial representation of information loaded from a source.

Documents may contain:

- Text content
- File information
- Metadata
- Source references

2.1.2 Nodes

Nodes are smaller units of information created from documents.

A large document can be divided into multiple nodes.

Each node may contain:

- Text content
- Unique identifier
- Metadata
- Relationships with other nodes
- Source document information

Nodes make document retrieval more efficient because the system can retrieve specific sections instead of entire documents.

2.1.3 Document Indexing

Indexing is the process of organizing documents into a structure that supports efficient retrieval.

LlamaIndex supports different indexing approaches, including:

- Vector Store Index
- Summary Index
- Tree Index
- Keyword Table Index

A vector index is commonly used for semantic similarity search.

2.1.4 Embedding Generation

LlamaIndex uses embedding models to convert nodes into numerical vectors.

The vectors represent semantic information and are used during similarity-based retrieval.

$$Node \rightarrow Embedding\ Vector$$

The generated vectors can be stored in compatible vector databases.

2.1.5 Vector Index

A vector index organizes nodes based on their embedding representations.

When a user submits a query, the system converts the query into an embedding and searches for similar nodes.

The most relevant nodes are selected and returned to the query engine.

2.1.6 Query Engine

The query engine is responsible for processing user queries and obtaining relevant information.

Its operations include:

1. Accepting a user query.
2. Searching the relevant index.
3. Retrieving matching nodes.
4. Combining retrieved information.
5. Generating an answer using an LLM.

2.2 Ingestion Pipeline

An important feature of LlamaIndex is its Ingestion Pipeline.
It applies a sequence of transformations to documents.
Typical transformations include:

- Text splitting
- Metadata extraction
- Node parsing
- Embedding generation
- Vector database insertion

An ingestion pipeline can also use caching to avoid repeating unchanged transformations.

2.3 Working Process of LlamaIndex

1. Load documents from different sources.
2. Convert documents into structured objects.
3. Divide documents into nodes.
4. Apply metadata extraction if required.
5. Generate embeddings for nodes.
6. Store nodes and embeddings in an index.
7. Accept a user query.
8. Retrieve relevant nodes from the index.
9. Generate a response using the retrieved context and an LLM.

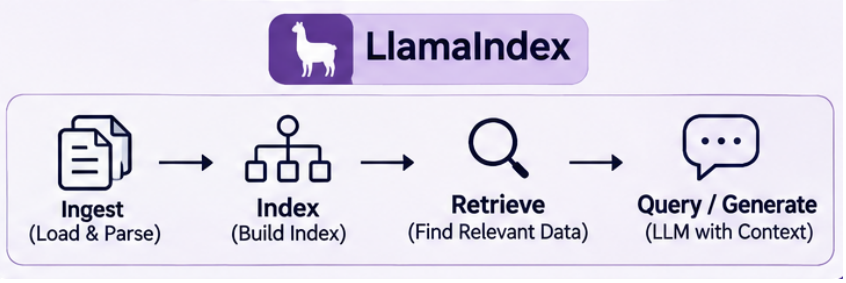


Figure 4: LamaIndex Overview

3 Haystack

Haystack is an open-source framework developed for building search systems, question-answering applications and Retrieval-Augmented Generation pipelines.

It provides modular components that can be connected to create complete information retrieval and generation workflows.

Haystack focuses on building structured, reusable and configurable pipelines.

3.1 Haystack Architecture

The general workflow is:

$$\boxed{\textit{Document Store} \rightarrow \textit{Embedder} \rightarrow \textit{Retriever} \rightarrow \textit{Results}}$$

3.1.1 Document Store

A Document Store is responsible for storing and managing documents.

It provides operations to:

- Insert documents.
- Retrieve documents.
- Update document information.
- Delete documents.
- Store embedding representations.

Examples of supported storage integrations include in-memory storage and external vector databases.

3.1.2 Embedder

An Embedder converts text or documents into numerical vector representations.

Haystack provides different embedding components for document and query processing.

Two important categories are:

Document Embedder:

Converts document content into embeddings during the indexing stage.

Text Embedder:

Converts user queries into embeddings during the retrieval stage.

3.1.3 Retriever

A Retriever searches the Document Store and returns documents relevant to the user's query.

Retrievers can use:

- Dense vector retrieval
- Sparse keyword retrieval
- Hybrid retrieval

The retrieved documents are passed to subsequent pipeline components.

3.1.4 Pipeline

A pipeline is a collection of connected components that execute a defined workflow.

Haystack pipelines can contain multiple components, branches and connections.

3.1.5 Generator

A Generator uses a language model to produce the final response.

It receives the user query and relevant retrieved documents.

It then generates an answer based on the provided context.

3.2 Indexing Pipeline

An indexing pipeline prepares documents for storage and retrieval.

Its typical operations are:

1. Document loading.
2. Document conversion.
3. Text preprocessing.
4. Embedding generation.
5. Document writing.
6. Storage in a Document Store.

3.3 Query Pipeline

The query pipeline processes user questions.
Its operations include:

1. Accept the user query.
2. Generate the query embedding.
3. Search the Document Store.
4. Retrieve relevant documents.
5. Construct a prompt.
6. Send the prompt to an LLM.
7. Return the generated answer.

3.4 Working Process of Haystack

1. Documents are collected from different sources.
2. Documents are processed and prepared.
3. Document embeddings are generated.
4. Documents are stored in a Document Store.
5. A query is received from the user.
6. The query embedder converts it into a vector.
7. The retriever searches for relevant documents.
8. The pipeline passes the retrieved documents to the generator.
9. The generator produces the final output.

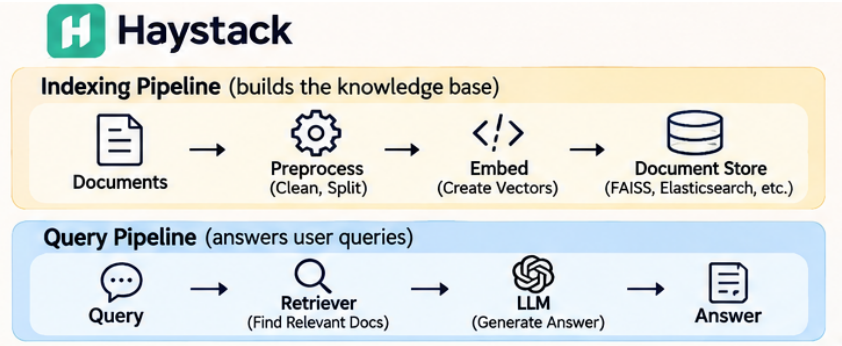


Figure 5: HayStack Overview

4 Key Differences

4.1 LangChain

LangChain focuses on connecting different components required to build LLM applications.

It is useful when developers need to integrate multiple models, tools, retrievers and workflows.

4.2 LlamaIndex

LlamaIndex focuses primarily on connecting LLMs with external data.

It provides specialized functionality for document ingestion, node creation, indexing and querying.

4.3 Haystack

Haystack focuses on constructing modular information retrieval and AI pipelines.

It provides components that can be connected to create indexing and query workflows.

Aspect	LangChain	LlamaIndex	Haystack
Primary Focus	General LLM application orchestration	Data ingestion & retrieval (RAG)	Production-ready NLP & RAG pipelines
Architecture	Modular components (chains, tools, agents)	Index-based (data → index → query)	Pipeline-based (indexing & query pipelines)
Strength	Flexibility and wide integrations	Efficient document indexing & retrieval	Robust, production-grade and highly configurable

Figure 6: Key differences